CIE-1 / TRANSPARENCY RECORD

AI Interaction Log

Tool: GitHub Copilot in VS Code **Dates:** 2026-09-27 to 2026-09-28

This log describes the actual assistance used for this project. The measurements and screenshots were collected from the local Omarchy Linux host; the assistant did not supply or invent the reported machine readings.

Selected prompts and responses

PROMPT 1 / INVESTIGATION SETUP

“I have a C program that walks through a large two-dimensional array, and I want to understand whether cache locality changes its runtime on Linux. Can you help me design a small reproducible investigation using real host measurements instead of assumed hardware specifications?”

Response: The assistant suggested recording the host CPU and memory context, capturing process and syscall snapshots, then compiling a benchmark that reads the same matrix in row-major and column-major order. It recommended saving the raw command output and benchmark rows so every reported observation could be checked later.

PROMPT 2 / BENCHMARK DESIGN

“Can you implement the comparison so both traversals process exactly the same initialized matrix, print a checksum to catch accidental differences, and avoid letting compiler vectorization hide the memory-access pattern?”

Response: The assistant wrote the row/column traversal experiment, compiled it with GCC using `-O2 -fno-tree-vectorize`, and ran three paired measurements with the first order alternated. It checked that both loop orders produced checksum `2097144125` and saved the rows in `evidence/locality.csv`. It advised describing the timing difference as specific to the machine and run conditions.

PROMPT 3 / MONITORING CHECK

“I’m adding a small localhost page to show the machine context and the busiest processes while the benchmark runs. The page currently calls the process column ‘current CPU percentage’; does the `ps` %CPU field really mean an instantaneous reading?”

Response: The assistant checked the installed `ps` manual instead of assuming the label was correct. It found that %CPU is CPU time divided by elapsed process running time, not an instantaneous sample, and changed the dashboard wording to “ordered by ps %CPU.” The supporting excerpt is saved as `evidence/ps_cpu_semantics.txt`.

AI error, correction, and evidence

AI interaction log / values verified against local command output

WHAT THE AI GOT WRONG

The first dashboard draft described its process ranking as “ordered by current CPU %.” That wording implies an instantaneous CPU sample. The dashboard actually obtains this field from `ps %CPU`, whose documented meaning is CPU time divided by elapsed process running time; it is not an instantaneous reading.

MACHINE EVIDENCE THAT DISPROVED IT

During validation, the assistant checked the installed `ps` manual and saved the relevant output in `evidence/ps_cpu_semantics.txt`. The local manual states:

```
%cpu / %CPU: CPU utilization of the process ... Currently, it is the CPU time used divided by
the time the process has been running (cputime/real-time ratio), expressed as a percentage.
```

The live output is independently reproducible with `ps -eo pid,stat,comm,%cpu --sort=-%cpu`; its captured rows and process states are in `evidence/process_snapshot.txt`. The assistant corrected the dashboard label to “ordered by ps %CPU.” It now names the source metric without claiming it is instantaneous. Recheck this command yourself during your demo.

AI contribution and verification

AI was used to draft the benchmark, dashboard, report structure, and this log. In this session, the assistant ran the Linux commands and C program on the host, checked that the row/column checksums match, opened the dashboard, called its status and benchmark endpoints, and verified the `ps` metric meaning using the installed manual. The student should rerun these checks, review the evidence, and personalize the report wording before submission.